

Refactoring “Understanding Dropout” coding question (Q5) of Homework 5

Kithmini Herath

1 Overview

This report details the refactoring of the `q_coding_dropout.ipynb` homework notebook. When I did this homework problem I noticed the lack of proper docstrings, type hinting, duplicated function definitions and quite a lengthy notebook due to the lack of modularity. Therefore, to transfer from a legacy academic notebook, originally designed purely for concept demonstration purposes, into a modular, robust codebase that adheres to modern Software Engineering and Machine Learning (ML) Engineering standards, I decided to refactor the code with the help of Claude Sonnet 4.5 in Claude Code. The codes are available at <https://github.com/KithminiHerath/CS282-HW5-Q5-Refactor>.

2 Method

The refactoring process followed a "Human-in-the-Loop" LLM-assisted workflow in Claude Code. First, I created a GitHub repository with the original `q_coding_dropout.ipynb` (student version) and `q_coding_dropout_sol.ipynb` (solution) notebooks and then locally prompted Claude Code in the repository directory. After Claude is done with its editions I will then manually inspect them in VSCode and proceed to do necessary editions/ improvements myself to reach a satisfactory level of code refactoring. I first started to edit the student version of the jupyter notebook and then transferred improved answers to this refactored version to create a separate solutions notebook. This notebook is configured to work on Google Colab.

2.1 Refactoring the student version

I prompted Claude as follows to obtain the initial version of the code:

```
Hi Claude, i have to refactor code in @q_coding_dropout.ipynb which is from a homework in a way that the code shouldn't lose any of its teaching value. i basically want it to follow good documented software engineering and ML Engineering processes. In particular i want you to follow PEP 8 style guidelines, PEP 484 type hinting and general machine learning and software engineering practices as outlined in Google's Python Style Guide and NumPy's Style Guide. please don't fill out any of the 'TODO:' sections because i still want this to remain as a coding exercise for students
```

Claude performed the initial standardization as follows:

- Adding PEP 484 [1] Type Hints to all functions (imports like `List`, `Optional`, `Tuple`).
- Implementing Google's Python Style Guide [2] type docstrings for classes like `ConvNet` and functions like `train_simple`.

- Standardizing formatting to PEP 8 [3] compliance, including import ordering and snake_case naming conventions.
- Adding inline comments explaining key operations.
- More descriptive plot labels and titles and clearer print statements with formatting.

while keeping the TODO sections empty as instructed.

After this pass I noticed that Claude included descriptive comments for the TODO sections that actually revealed parts of the answers for the written portions of the assignment. Therefore, after inspecting all these I manually deleted such comments to maintain the pedagogical integrity. Moreover, the function `init_with_dropout` was defined at two different places, probably with the intent to allow the student to run different sections of the notebook without having to run everything from the beginning. This is when I decided that this code would benefit from modularization so that all necessary functions can be easily imported at the very beginning of the notebook.

I further conducted the following architectural refactoring with Claude's help when needed:

1. To prevent duplicate use of plotting code segments in the notebook, I separated the training logic from plotting logic to adhere to the Single Responsibility Principle. Claude improved the plots by adding proper captions, figure size formatting, and axis labelling.
2. Created a library structure (`dropout_lib`) with `trainer.py`, `utils.py`, `vis_utils.py` files for reusable components.
 - `trainer.py`: Handles the training loops.
 - `vis_utils.py`: Contains all reusable plotting logic. This allows visualization to be updated independently of training logic. The train functions access these to visualize training curves.
 - `utils.py`: Houses helper functions like `set_seed` and `add_cheating_feature`
3. Implemented global seed setting for reproducibility between runs.
 - Sets seeds for `random`, `numpy` and `torch`.
 - The seed was set at the very beginning of the notebook as well as to the last CIFAR10 training cells, so that the student can run those cells and iterate through training calls without losing reproducibility and also for fair comparison of training across the two different training setups (`model_dropout`, `model_no_dropout`).

Once this notebook was fully refactored, I moved to the solutions notebook by creating a copy of the this refactored notebook.

2.2 Improving the solutions notebook

First, I pulled the codebase's github repository to Colab and configured the solutions notebook to run in a Colab environment. I previously did special participation B for the coding problems of this same homework with Claude Sonnet 4.5. So for this section I basically compared the solutions from Claude and the solutions provided by staff and started filling the TODO sections with the solutions that were clear, simple, and had better explanations of the steps followed. I ran this solutions notebook and was able to go through the entire assignment with no errors while obtaining results similar to the solutions provided by the staff. This test run validated the seamless usage of this codebase and I updated the github repository with the final versions of the refactored student version and solutions of the notebook.

3 Conclusion

For this refactoring assignment, Claude did a great job in one-shot standardizing the code initially according to software engineering and ML engineering standards. Then, with my manual editions and feedback to Claude I was able to do further code architectural improvements refactor the notebook to its final version. This notebook was only tested in a Google Colab environment. To run this locally, students may have to comment out the Google Colab setting up code cells and run this in a Python environment with Python 3.7+ and PyTorch 1.7+. However, I haven't tested this locally and leave it for future work. I believe that modularizing the code would make it easier for future students to follow the notebooks as a platform for seamless high-level exploration of the underlying ML concepts.

References

- [1] Guido Van Rossum, Jukka Lehtosalo, and Łukasz Langa. PEP 484 – Type Hints. <https://peps.python.org/pep-0484/>.
- [2] Google. Google Python Style Guide. <https://google.github.io/styleguide/pyguide.html>.
- [3] Guido Van Rossum, Barry Warsaw, and Nick Coghlan. PEP 8 – Style Guide for Python Code. <https://peps.python.org/pep-0008/>.